

Contents

Les Chaînes de caractères en Python (str)	3
1 Nature des chaînes	4
2 Création d'une chaîne	4
3 Caractères d'échappement (<i>ajout</i>)	4
4 Raw strings — r"""	5
Point important	5
5 Immutabilité — démonstration fondamentale	5
6 Indexation	6
7 Slicing — mécanisme central	6
8 Parcours d'une chaîne	7
9 Concaténation et répétition	7
Longueur	7
1 1 Méthodes de transformation	7
1 2 Suppression d'espaces	8
1 3 Remplacement	8
1 4 Tests logiques	8
1 5 Recherche	9
1 6 Découpage et assemblage	9
1 7 Formatage moderne — f-strings	9
1 8 Ord et chr	10
1 9 Normalisation des espaces	10
2 0 Palindrome simplifié	11
2 1 Performance — piège concaténation	11
2 2 Cas métier : Nettoyage d'un nom utilisateur	11

2 3 Mini-exercices de production (<i>ajout</i>)	12
Exercice 1 — Prédiction	12
Exercice 2 — Production	12
Exercice 3 — Débogage	13
Synthèse opérationnelle	13
Tableau récapitulatif des méthodes (<i>ajout</i>)	13
20 Questions intelligentes — Les Chaînes en Python	14
1) Pourquoi une chaîne est immuable ?	14
Énoncé	14
Solution	14
2) Différence entre find et index	15
Énoncé	15
Solution	15
3) Pourquoi la concaténation répétée est déconseillée ?	15
Énoncé	15
Solution	16
4) Différence entre strip, lstrip et rstrip	16
Énoncé	16
Solution	16
5) Pourquoi split sans argument supprime espaces multiples ?	17
Énoncé	17
Solution	17
6) Comment vérifier qu'une chaîne contient uniquement des chiffres ?	17
Énoncé	17
Solution	17
7) Pourquoi lower() ne modifie pas la chaîne originale ?	18
Énoncé	18
Solution	18
8) Comment inverser proprement une chaîne ?	18
Énoncé	18
Solution	18
9) Différence entre replace et re.sub	19
Énoncé	19
Solution	19

10) Comment normaliser les espaces multiples ?	19
Énoncé	19
Solution	19
11) Différence entre startswith et in	20
Énoncé	20
Solution	20
12) Pourquoi f-string est préférable ?	20
Énoncé	20
Solution	20
13) Comment supprimer les accents ?	21
Énoncé	21
Solution	21
14) Comment compter occurrences d'un mot ?	21
Énoncé	21
Solution	21
15) Comment vérifier un email simple ?	22
Énoncé	22
Solution	22
16) Pourquoi join est plus rapide que + ?	22
Énoncé	22
Solution	22
17) Comment sécuriser une entrée utilisateur ?	23
Énoncé	23
Solution	23
18) Pourquoi comparer des chaînes est sensible à la casse ?	23
Énoncé	23
Solution	23
19) casefold vs lower	24
Énoncé	24
Solution	24
20) Cas métier complet : génération d'identifiant	24
Énoncé	24
Solution	24

Les Chaînes de caractères en Python (str)

Le support est une propriété de **Bechir Bejaoui**

1 Nature des chaînes

Une chaîne :

- est une séquence ordonnée de caractères
 - est immuable
 - est indexable
 - est itérable
 - supporte slicing et méthodes dédiées
-

2 Création d'une chaîne

```
"""
Une chaîne peut être délimitée par :
- guillemets simples
- guillemets doubles
- triples guillemets pour multi-ligne
"""
```

```
s1 = "Python"
s2 = 'Python'
s3 = """Ligne 1
Ligne 2"""
```

```
print(s1)
print(s3)
```

3 Caractères d'échappement (*ajout*)

```
"""
Les séquences d'échappement permettent d'insérer
des caractères spéciaux dans une chaîne.
"""
```

```
print("Ligne 1\nLigne 2")      # saut de ligne
print("Col1\tCol2")           # tabulation
```

```
print("Il dit : \"Bonjour\"")    # guillemet dans chaîne
print("C:\\Users\\alice")        # antislash littéral
```

Sortie :

```
Ligne 1
Ligne 2
Col1    Col2
Il dit : "Bonjour"
C:\Users\alice
```

4 Raw strings — r"""

```
"""
Une raw string désactive les séquences d'échappement.
Indispensable pour les chemins Windows et les expressions régulières.
"""
```

```
chemin = r"C:\Users\alice\Documents"
print(chemin)
```

```
pattern = r"\d+\.\d+"
print(pattern)
```

Point important

Sans le préfixe `r`, `\U`, `\a`, `\d` seraient interprétés comme des échappements.

5 Immutabilité — démonstration fondamentale

```
"""
Les chaînes sont immuables.
On ne peut pas modifier un caractère en place.
Toute modification crée un nouvel objet.
"""
```

```
mot = "Hello"

print(id(mot))

mot = mot + " world"

print(id(mot))    # adresse différente
```

Tentative interdite :

```
"""
Modification directe impossible.
"""
```

```
mot = "Hello"
```

```
try:
    mot[0] = "Y"
except TypeError as e:
    print("Erreur :", e)
```

6 Indexation

```
"""
Indexation positive et négative.
"""
```

```
texte = "Python"
```

```
print(texte[0])    # P
print(texte[-1])   # n
```

7 Slicing — mécanisme central

Syntaxe : chaîne[start:stop:step]

```
"""
Le slicing permet d'extraire des sous-chaînes.
La borne stop est exclusive.
"""
```

```
texte = "Python"
```

```
print(texte[0:3])    # Pyt
print(texte[:3])     # Pyt
print(texte[3:])     # hon
print(texte[::-1])   # inversion
```

8 Parcours d'une chaîne

```
"""
Une chaîne est itérable.
"""

for caractere in "DevOps":
    print(caractere)
```

9 Concaténation et répétition

```
"""
+ concatène
* répète
"""

a = "Dev"
b = "Ops"

print(a + b)
print("-" * 10)
```

Longueur

```
"""
len retourne le nombre de caractères.
"""

print(len("Python"))
```

1 1 Méthodes de transformation

Ces méthodes créent une nouvelle chaîne.

```
"""
Transformation de casse.
"""

s = "hELLO wORLD"
```

```
print(s.lower())
print(s.upper())
print(s.capitalize())
print(s.title())
print(s.swapcase())
```

1 2 Suppression d'espaces

```
"""
strip enlève au début et à la fin.
rstrip enlève au début uniquement.
rstrip enlève à la fin uniquement.
"""
```

```
chaine = "   Python   "

print("|" + chaine.strip() + "|")
print("|" + chaine.lstrip() + "|")
print("|" + chaine.rstrip() + "|")
```

1 3 Remplacement

```
"""
replace modifie toutes les occurrences.
"""
```

```
phrase = "Python est puissant"

print(phrase.replace("puissant", "rapide"))
```

1 4 Tests logiques

```
"""
Méthodes de validation.
"""
```

```
print("abc123".isalnum())
print("123".isdigit())
```



```
print("abc".isalpha())
print("HELLO".isupper())
```

1 5 Recherche

```
"""
find retourne -1 si absent.
index lève une exception.
"""
```

```
texte = "Python DevOps"

print(texte.find("Dev"))
print(texte.count("o"))
```

1 6 Découpage et assemblage

```
"""
split transforme en liste.
join reconstruit une chaîne.
"""
```

```
phrase = "Python DevOps Cloud"

mots = phrase.split()
print(mots)

reconstruit = "-".join(mots)
print(reconstruit)
```

1 7 Formatage moderne — f-strings

```
"""
f-strings permettent interpolation et formatage.
"""
```

```
nom = "Alice"
age = 30
```

```

print(f"Nom: {nom}, Age: {age}")

Format numérique :

"""
Formatage avec précision décimale.
"""

prix = 19.9876
print(f"{prix:.2f}")

Formatage avancé — alignement et padding (ajout) :

"""
Alignement et remplissage pour affichages tabulaires.
"""

print(f"{'Produit':<15} {'Prix':>8}")
print(f"{'Laptop':<15} {999.99:>8.2f}")
print(f"{'Souris':<15} {29.90:>8.2f}")

# Zéro padding sur entiers
numero = 42
print(f"Référence : {numero:05d}")    # 00042

```

1 8 Ord et chr

```

"""
Conversion caractère <-> code Unicode.
"""

print(ord("A"))
print(chr(65))

```

1 9 Normalisation des espaces

```

"""
Supprimer les espaces multiples.
"""

phrase = "Python   DevOps   Cloud"

```

```
normalisee = " ".join(phrase.split())
print(normalisee)
```

2 0 Palindrome simplifié

```
"""
Comparaison avec inversion.
"""
```

```
mot = "radar"

print(mot == mot[::-1])
```

2 1 Performance — piège concaténation

```
"""
Concaténer dans une boucle crée de nouveaux objets.
Préférer join.
"""
```

```
elements = ["a", "b", "c"]

resultat = ""
for e in elements:
    resultat += e

print(resultat)

print("".join(elements))
```

2 2 Cas métier : Nettoyage d'un nom utilisateur

Objectif :

- supprimer espaces inutiles
- supprimer accents
- passer en minuscules
- générer identifiant propre

```

"""
Nettoyage robuste pour identifiant utilisateur.
"""

import unicodedata

def nettoyer_nom(nom):
    nom = nom.strip()
    nom = unicodedata.normalize("NFKD", nom)
    nom = "".join(c for c in nom if not unicodedata.combining(c))
    nom = nom.lower()
    nom = nom.replace(" ", "")
    return nom

print(nettoyer_nom(" BéChir Bejaoui "))

```

2 3 Mini-exercices de production (*ajout*)

Exercice 1 — Prédiction

Prédire la sortie avant exécution :

```

texte = " Python DevOps "

print(texte.strip())
print(texte.strip().split())
print("-".join(texte.strip().split()))
print(texte.strip()[::-1])

```

Exercice 2 — Production

Écrire un programme qui :

1. Prend la chaîne " bonjour MONDE "
2. Supprime les espaces inutiles
3. Met en title case
4. Remplace "Monde" par "Python"
5. Affiche le résultat final

Votre solution ici

Solution

```

chaîne = " bonjour MONDE "

```

```
resultat = chaine.strip().title().replace("Monde", "Python")
print(resultat)
```

Exercice 3 — Débogage

Ce code contient 2 erreurs. Les identifier et les corriger.

```
nom = " Alice Dupont "
nom[0] = "a"
identifiant = nom.strip().lower().replace(" ", "_")
print(identifiant)
```

Solution

```
# Erreur 1 : modification directe impossible sur une chaîne immuable
# Erreur 2 : strip() doit être appelé avant lower() pour nettoyer d'abord les espaces

nom = " Alice Dupont "
identifiant = nom.strip().lower().replace(" ", "_")
print(identifiant)
```

Synthèse opérationnelle

Une chaîne :

- est immuable
 - supporte les caractères d'échappement (`\n`, `\t`, `\\`)
 - utilise `r""` pour désactiver les échappements (chemins, regex)
 - supporte slicing puissant
 - possède de nombreuses méthodes intégrées
 - doit être manipulée efficacement (join > concaténation répétée)
 - est centrale dans le traitement de données, parsing, validation, logs
-

Tableau récapitulatif des méthodes (*ajout*)

Méthode	Rôle	Exemple
<code>lower()</code>	Minuscules	<code>"ABC".lower()</code> → <code>"abc"</code>
<code>upper()</code>	Majuscules	<code>"abc".upper()</code> → <code>"ABC"</code>

Méthode	Rôle	Exemple
<code>title()</code>	Première lettre maj	<code>"hello world".title() → "Hello World"</code>
<code>strip()</code>	Supprime espaces début/fin	<code>" abc ".strip() → "abc"</code>
<code>lstrip()</code>	Supprime espaces début	<code>" abc ".lstrip() → "abc "</code>
<code>rstrip()</code>	Supprime espaces fin	<code>" abc ".rstrip() → " abc"</code>
<code>replace(a,b)</code>	Remplace occurrences	<code>"ab".replace("a","x") → "xb"</code>
<code>split()</code>	Découpe en liste	<code>"a b".split() → ["a","b"]</code>
<code>join()</code>	Assemble une liste	<code>"-".join(["a","b"]) → "a-b"</code>
<code>find()</code>	Cherche, retourne -1	<code>"abc".find("b") → 1</code>
<code>index()</code>	Cherche, lève exception	<code>"abc".index("b") → 1</code>
<code>count()</code>	Compte occurrences	<code>"abab".count("a") → 2</code>
<code>startswith()</code>	Teste début	<code>"abc".startswith("a") → True</code>
<code>endswith()</code>	Teste fin	<code>"abc".endswith("c") → True</code>
<code>isdigit()</code>	Que des chiffres ?	<code>"123".isdigit() → True</code>
<code>isalpha()</code>	Que des lettres ?	<code>"abc".isalpha() → True</code>
<code>casefold()</code>	Minuscules Unicode	<code>"ß".casefold() → "ss"</code>

20 Questions intelligentes — Les Chaînes en Python

Format : Énoncé → Solution complète avec démarche explicative et démonstration observable

1) Pourquoi une chaîne est immuable ?

Énoncé

Démontrer que toute modification crée un nouvel objet.

Solution

```
"""
Explication
Les chaînes sont immuables.
Toute opération qui semble modifier la chaîne crée en réalité
un nouvel objet en mémoire.
On le vérifie via id().
"""
```

```

"""

texte = "Python"

print("ID initial :", id(texte))

texte = texte + " DevOps"

print("Nouvelle valeur :", texte)
print("Nouvel ID :", id(texte))

```

2) Différence entre find et index

Énoncé

Que se passe-t-il si le motif n'est pas trouvé ?

Solution

```

"""
Explication
find retourne -1 si le motif est absent.
index lève une exception.
Utiliser find est plus sûr en production.
"""

texte = "Python"

print("find:", texte.find("Dev"))

try:
    print("index:", texte.index("Dev"))
except ValueError as e:
    print("Erreur index:", e)

```

3) Pourquoi la concaténation répétée est déconseillée ?

Énoncé

Comparer concaténation en boucle vs join.

Solution

```
"""
Explication
Chaque concaténation crée un nouvel objet.
join est optimisé et plus performant.
"""

elements = ["A", "B", "C", "D"]

resultat = ""
for e in elements:
    resultat += e

print("Concaténation :", resultat)

optimise = "".join(elements)
print("Join :", optimise)
```

4) Différence entre strip, lstrip et rstrip

Énoncé

Observer leur comportement respectif.

Solution

```
"""
Explication
strip enlève début et fin.
lstrip enlève au début.
rstrip enlève à la fin.
"""

chaine = "  Python  "

print("|" + chaine.strip() + "|")
print("|" + chaine.lstrip() + "|")
print("|" + chaine.rstrip() + "|")
```

5) Pourquoi split sans argument supprime espaces multiples ?

Énoncé

Comprendre le comportement implicite.

Solution

```
"""
Explication
split() sans argument considère tout espace blanc comme séparateur
et ignore les répétitions.
"""
```

```
phrase = "Python  DevOps  Cloud"

print(phrase.split())
```

6) Comment vérifier qu'une chaîne contient uniquement des chiffres ?

Énoncé

Comparer isdigit et isnumeric.

Solution

```
"""
Explication
isdigit vérifie chiffres simples.
isnumeric accepte plus de caractères numériques Unicode.
"""
```

```
print("123".isdigit())
print("123".isnumeric())
print("2".isdigit())
print("2".isnumeric())
```

7) Pourquoi lower() ne modifie pas la chaîne originale ?

Énoncé

Démontrer immutabilité via transformation.

Solution

```
"""
Explication
Les méthodes retournent une nouvelle chaîne.
L'originale reste intacte.
"""
```

```
s = "HELLO"

print("Original :", s)
print("Transformé :", s.lower())
print("Après :", s)
```

8) Comment inverser proprement une chaîne ?

Énoncé

Utiliser slicing.

Solution

```
"""
Explication
[::-1] inverse la chaîne grâce au pas négatif.
"""
```

```
mot = "Python"

print(mot[::-1])
```

9) Différence entre replace et re.sub

Énoncé

Quand replace ne suffit pas ?

Solution

```
"""
Explication
replace fonctionne sur texte fixe.
re.sub permet expressions régulières.
"""

import re

texte = "Prix: 100 TND"

print(texte.replace("100", "200"))

print(re.sub(r"\d+", "200", texte))
```

10) Comment normaliser les espaces multiples ?

Énoncé

Transformer en espace simple.

Solution

```
"""
Explication
split puis join reconstruit avec un espace unique.
"""

phrase = "Python    DevOps    Cloud"

normalisee = " ".join(phrase.split())

print(normalisee)
```

11) Différence entre startswith et in

Énoncé

Comparer précision des tests.

Solution

```
"""
Explication
startswith vérifie début précis.
in vérifie présence n'importe où.
"""

texte = "Python DevOps"

print(texte.startswith("Python"))
print("Dev" in texte)
```

12) Pourquoi f-string est préférable ?

Énoncé

Comparer trois méthodes.

Solution

```
"""
Explication
f-string est plus lisible et plus performant.
"""

nom = "Alice"
age = 30

print("Nom: " + nom + ", Age: " + str(age))
print("Nom: {}, Age: {}".format(nom, age))
print(f"Nom: {nom}, Age: {age}")
```

13) Comment supprimer les accents ?

Énoncé

Nettoyage robuste.

Solution

```
"""
Explication
unicodedata permet normalisation Unicode.
On enlève les caractères combinés.
"""

import unicodedata

texte = "BéChir"

normalise = unicodedata.normalize("NFKD", texte)
sans_accent = "".join(c for c in normalise if not unicodedata.combining(c))

print(sans_accent)
```

14) Comment compter occurrences d'un mot ?

Énoncé

Comparer count et regex.

Solution

```
"""
Explication
count est simple mais ne gère pas variations.
regex permet flexibilité.
"""

import re

texte = "Python python PYTHON"

print(texte.lower().count("python"))

print(len(re.findall(r"python", texte, re.IGNORECASE)))
```

15) Comment vérifier un email simple ?

Énoncé

Validation minimale.

Solution

```
"""
Explication
Validation simple via expression régulière.
Pas validation RFC complète.
"""

import re

email = "user@test.com"

pattern = r"^\w\.-]+@[\w\.-]+\.\w+$"

print(bool(re.match(pattern, email)))
```

16) Pourquoi join est plus rapide que + ?

Énoncé

Observer logique interne.

Solution

```
"""
Explication
join alloue une seule fois la mémoire.
+ recrée la chaîne à chaque opération.
"""

mots = ["Dev", "Ops", "Cloud"]

print("".join(mots))
```

17) Comment sécuriser une entrée utilisateur ?

Énoncé

Nettoyer avant traitement.

Solution

```
"""
Explication
strip + lower + validation évitent erreurs.
"""

entree = "  ADMIN  "

nettoyee = entree.strip().lower()

print(nettoyee)
```

18) Pourquoi comparer des chaînes est sensible à la casse ?

Énoncé

Démonstration.

Solution

```
"""
Explication
La comparaison est sensible à la casse.
Utiliser lower() ou casefold().
"""

print("Python" == "python")

print("Python".lower() == "python".lower())
```

19) casefold vs lower

Énoncé

Pourquoi casefold est plus robuste ?

Solution

```
"""
Explication
casefold est plus agressif pour comparaisons Unicode.
Recommandé pour comparaisons internationales.
"""

print("ß".lower())
print("ß".casefold())
```

20) Cas métier complet : génération d'identifiant

Énoncé

Transformer un nom complet en identifiant système sécurisé.

Solution

```
"""
Explication
Enchaînement complet :
1. strip
2. normalisation Unicode
3. suppression accents
4. lower
5. remplacement espaces
6. suppression caractères non alphanumériques
"""

import unicodedata
import re

def generer_identifiant(nom):
    nom = nom.strip()
    nom = unicodedata.normalize("NFKD", nom)
    nom = "".join(c for c in nom if not unicodedata.combining(c))
    nom = nom.lower()
```



```
nom = nom.replace(" ", "_")
nom = re.sub(r"[^a-z0-9_]", "", nom)
return nom

print(generer_identifiant(" BéChir Bejaoui "))
```